

# AWS API Gateway Lab Part 1

## To Begin with the Lab

### Summary of the Lab

This lab built a CRUD backend using **Amazon DynamoDB** and **AWS Lambda**. First, a UsersTable table was created with userId as the partition key, then an **IAM** role named LambdaDynamoDBExecutionRole was prepared so Lambda could access DynamoDB. After that, three separate Lambda functions were created: CreateUserFunction, GetUserFunction, and DeleteUserFunction, each using Python 3.12 and the same execution role. The create function was tested by inserting a sample user record, the get function was tested for both existing and missing users, and the delete function was tested by removing the record and confirming it disappeared from the table. This prepared the backend for the next step, which was API Gateway integration.

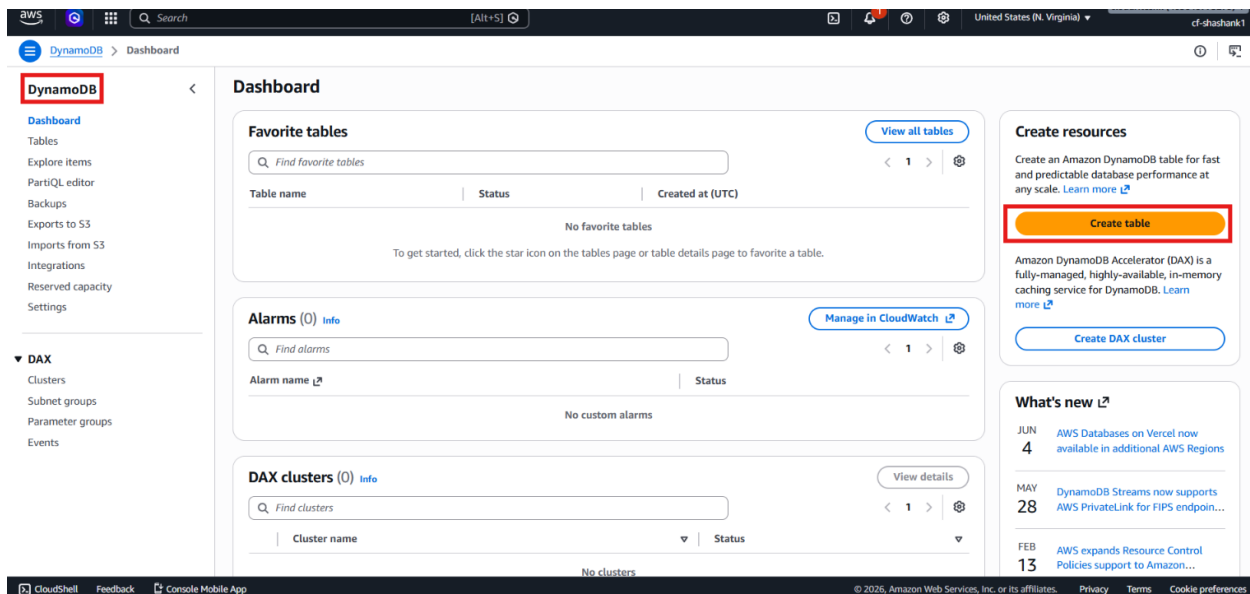
### Understanding the Lab

This lab shows how to build the backend layer for a simple user-management application using **AWS Lambda** and **Amazon DynamoDB**. A front-end application should not talk directly to DynamoDB, so the logic is split into separate Lambda functions for create, read, and delete operations. Using one dedicated IAM role for all three functions keeps permissions controlled and reusable. This pattern is a clean way to implement separation of concerns and makes the backend easier to test and maintain.

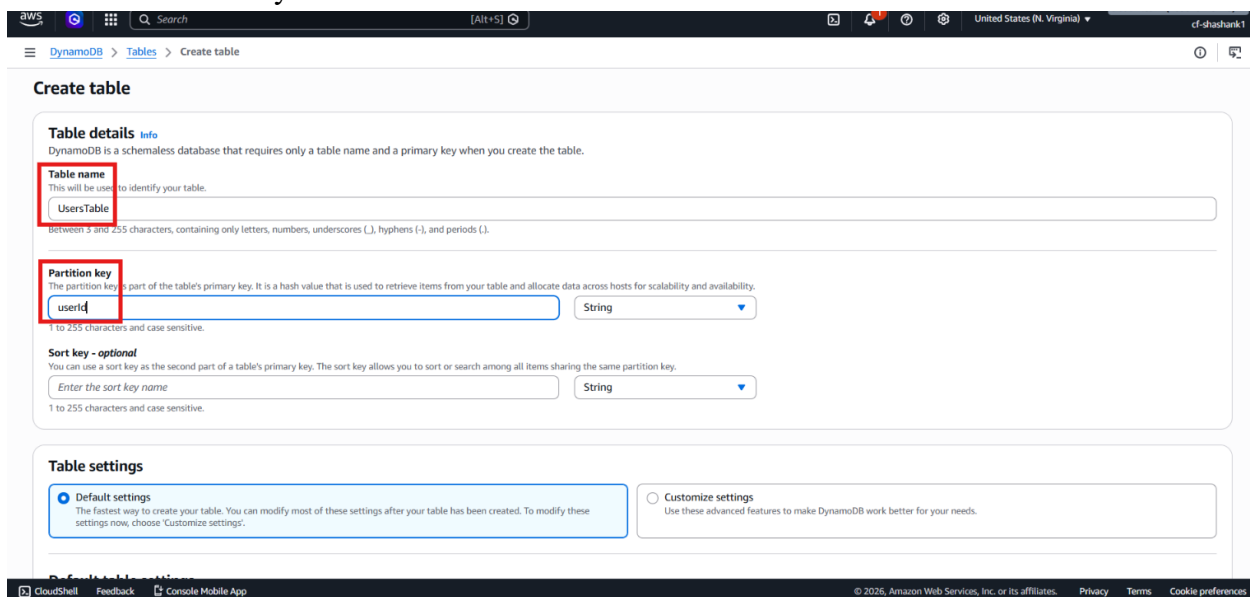
### Lab Steps

#### Step 1 – Create the DynamoDB Backend Table

- Sign in to the AWS Management Console.
- Open **Amazon DynamoDB**.
- Click **Create table**.



- Enter the table name: UsersTable
- Set the partition key name to userId.
- Set the partition key type to String.
- Leave the sort key blank.



- Click **Create table**.

## Step 2 – Understand the API Requirement

- Review the lab goal.
- Note that the front-end application cannot directly access DynamoDB.
- Understand that an API layer is needed for:
  - Creating a user
  - Fetching a user

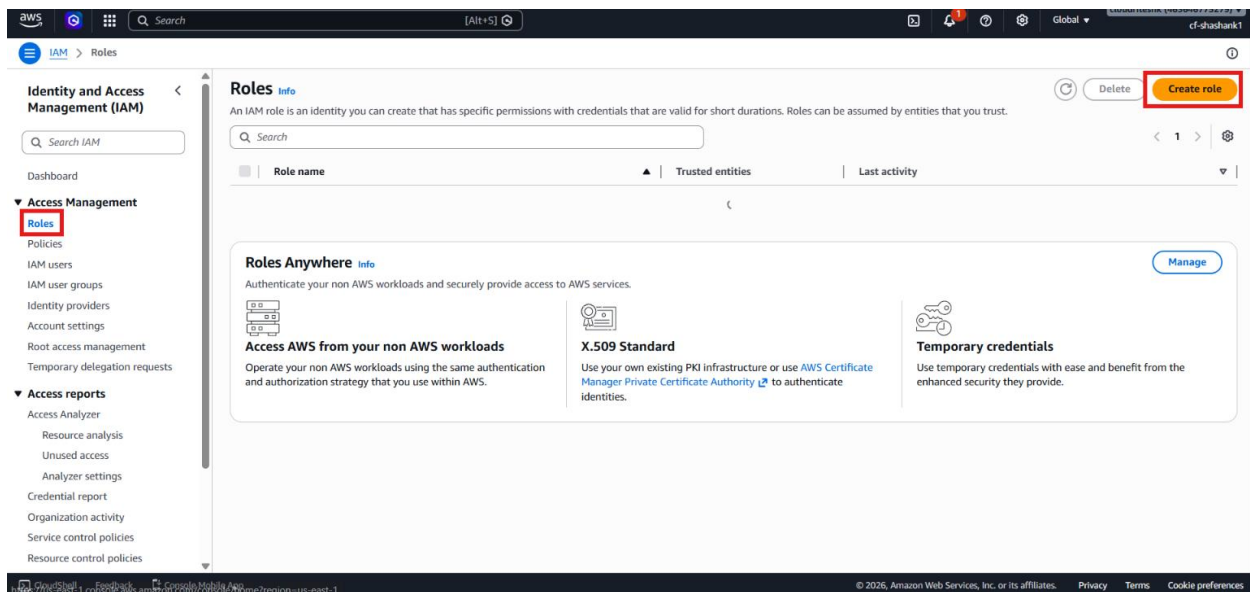
- Deleting a user
- Understand that the API logic will be hosted in **AWS Lambda**.

### Step 3 – Plan the Lambda Functions

- Decide to create three separate Lambda functions:
  - createUserFunction
  - getUserFunction
  - deleteUserFunction
- Keep each function focused on one task.
- Follow separation of concerns for easier testing and maintenance.

### Step 4 – Create the IAM Role for Lambda

- Open IAM → Roles.
- Click **Create role**.



- Choose **AWS service** as the trusted entity type.
- Select **Lambda**.

Step 2: Add permissions

**Trusted entity type**

- ☒ **AWS service**  
Allow AWS services like EC2, Lambda, or others to perform actions in this account.
- ☐ AWS account  
Allow entities in other AWS accounts belonging to you or a 3rd party to perform actions in this account.
- ☐ Web identity  
Allows users federated by the specified external web identity provider to assume this role to perform actions in this account.
- ☐ SAML 2.0 federation  
Allow users federated with SAML 2.0 from a corporate directory to perform actions in this account.
- ☐ Custom trust policy  
Create a custom trust policy to enable others to perform actions in this account.

**Use case**  
Allow an AWS service like EC2, Lambda, or others to perform actions in this account.

**Service or use case**  
Lambda

Choose a use case for the specified service.

**Use case**  
☒ **Lambda**  
Allow Lambda functions to call AWS services on your behalf.

☐ AWSServiceRoleForLambda  
Allows Lambda to manage AWS resources on your behalf.

Cancel **Next**

- Attach AmazonDynamoDBFullAccess.
- Enter the role name: LambdaDynamoDBExecutionRole

Step 2: Add permissions

**Add permissions**

Permissions policies (1/2300)

Choose one or more policies to attach to your new role.

Filter by Type: All types (4 matches)

| Policy name                                                       | Type        | Description                                   |
|-------------------------------------------------------------------|-------------|-----------------------------------------------|
| <input checked="" type="checkbox"/> AmazonDynamoDBFullAccess      | AWS managed | Provides full access to Amazon DynamoDB...    |
| <input type="checkbox"/> AmazonDynamoDBFullAccess_v2              | AWS managed | Provides full access to Amazon DynamoDB       |
| <input type="checkbox"/> AmazonDynamoDBFullAccesswithDataPipeline | AWS managed | This policy is on a deprecation path. See ... |
| <input type="checkbox"/> AmazonDynamoDBReadOnlyAccess             | AWS managed | Provides read only access to Amazon Dyn...    |

► Set permissions boundary - optional

Cancel Previous **Next**

- Create the role.
- Save this role for all three Lambda functions.

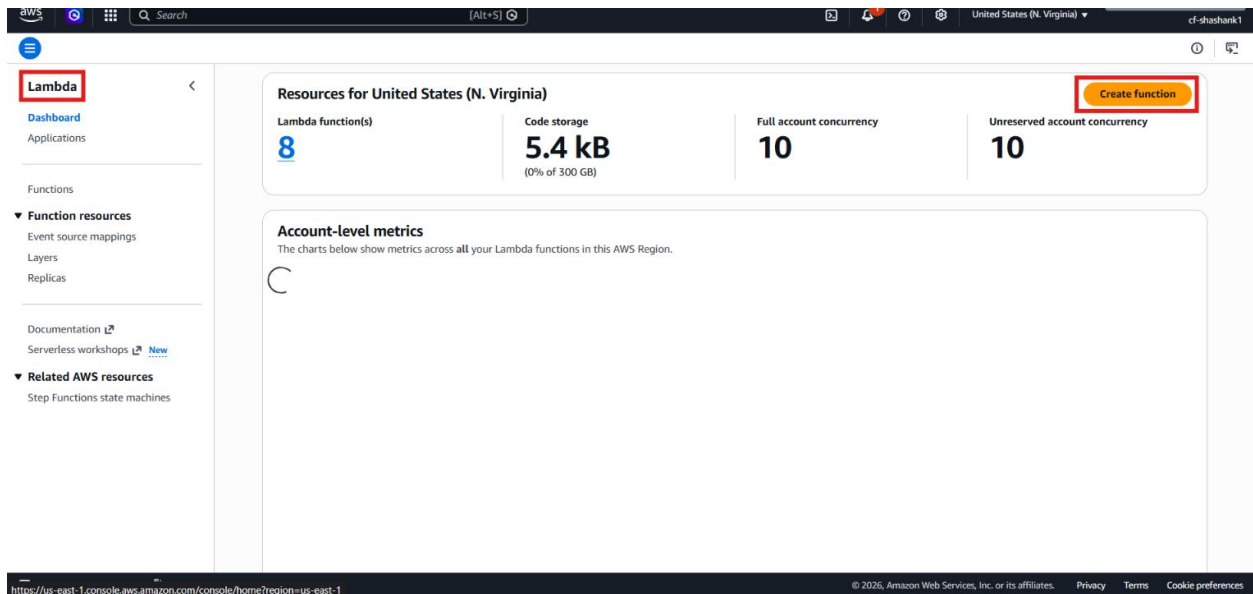
## Step 5 – Confirm the Prerequisites Are Ready

- Verify that the **UsersTable** DynamoDB table exists.
- Verify that the **LambdaDynamoDBExecutionRole** has been created.
- Confirm that Lambda now has permission to talk to DynamoDB.
- Prepare to create the HTTP API in the next part of the lab.

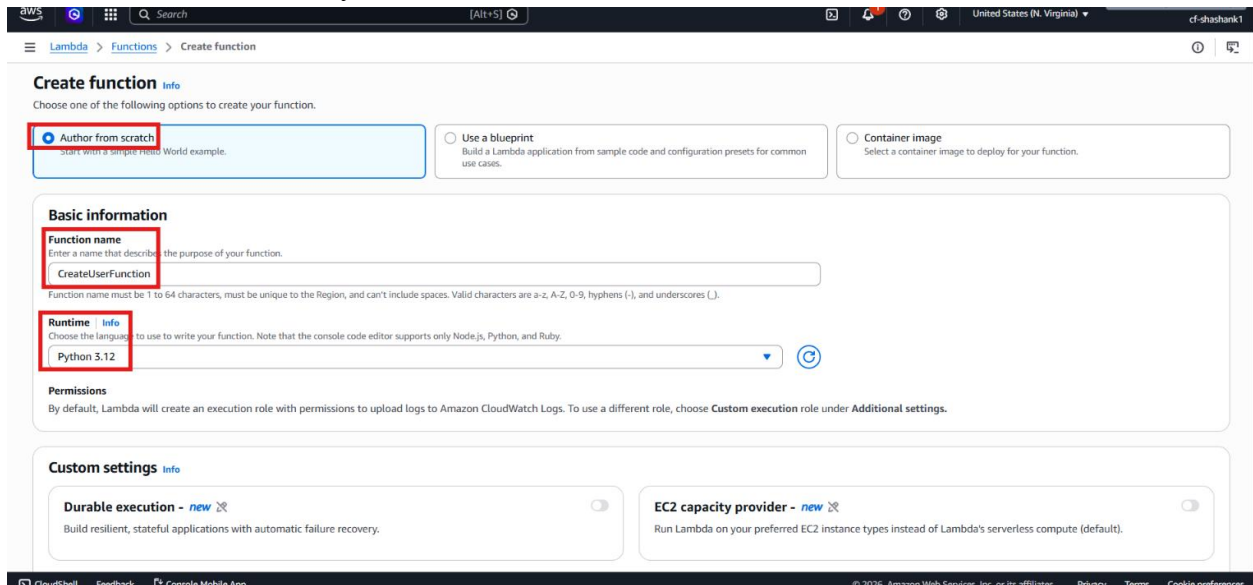
## Step 6 – Create the First Lambda Function

- Sign in to the AWS Management Console.

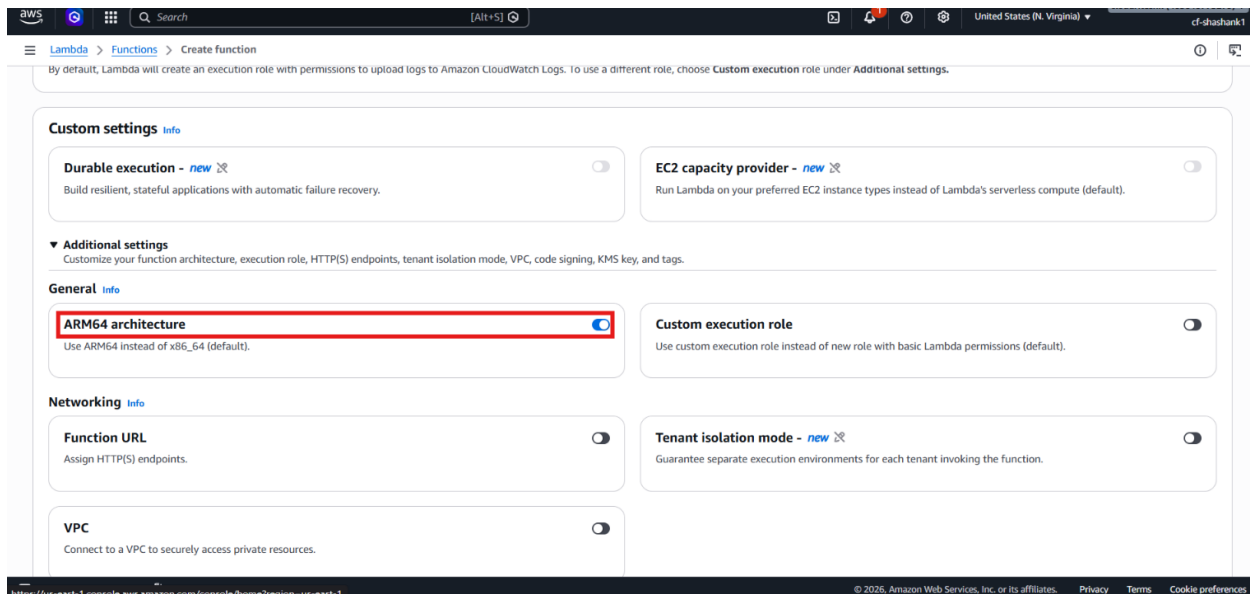
- Open **AWS Lambda**.
- Click **Create function**.



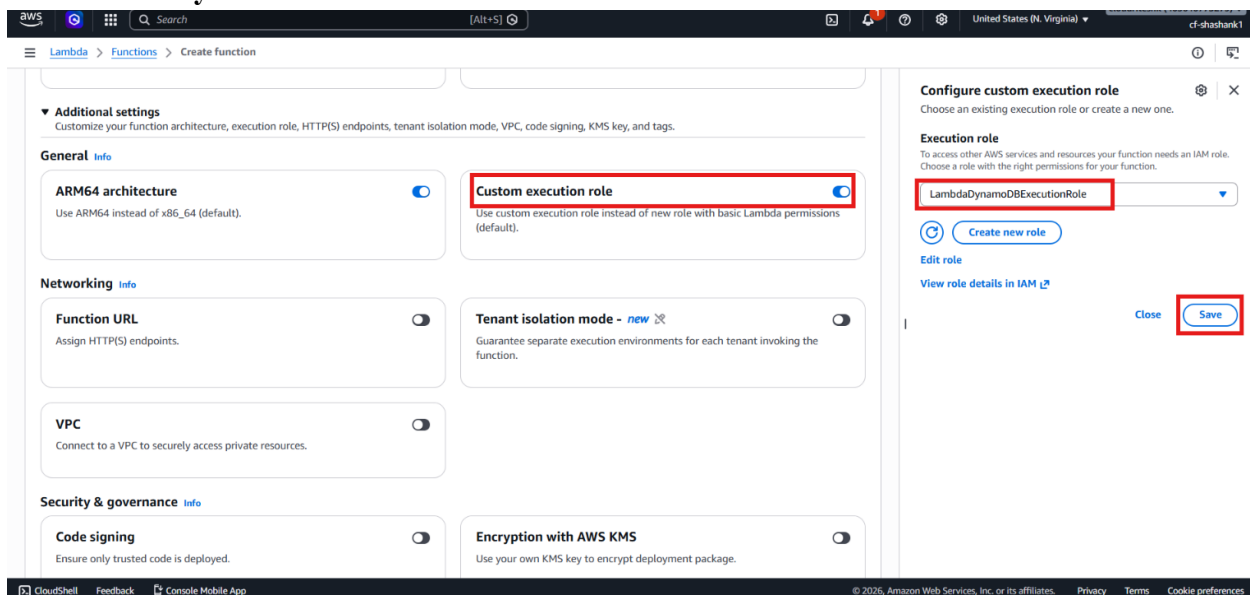
- Select **Author from scratch**.
- Enter the function name: **CreateUserFunction**
- Choose the runtime: **Python 3.12**



- Under Custom Settings → Additional Settings, enable **ARM64 architecture** to use **x86\_64**.



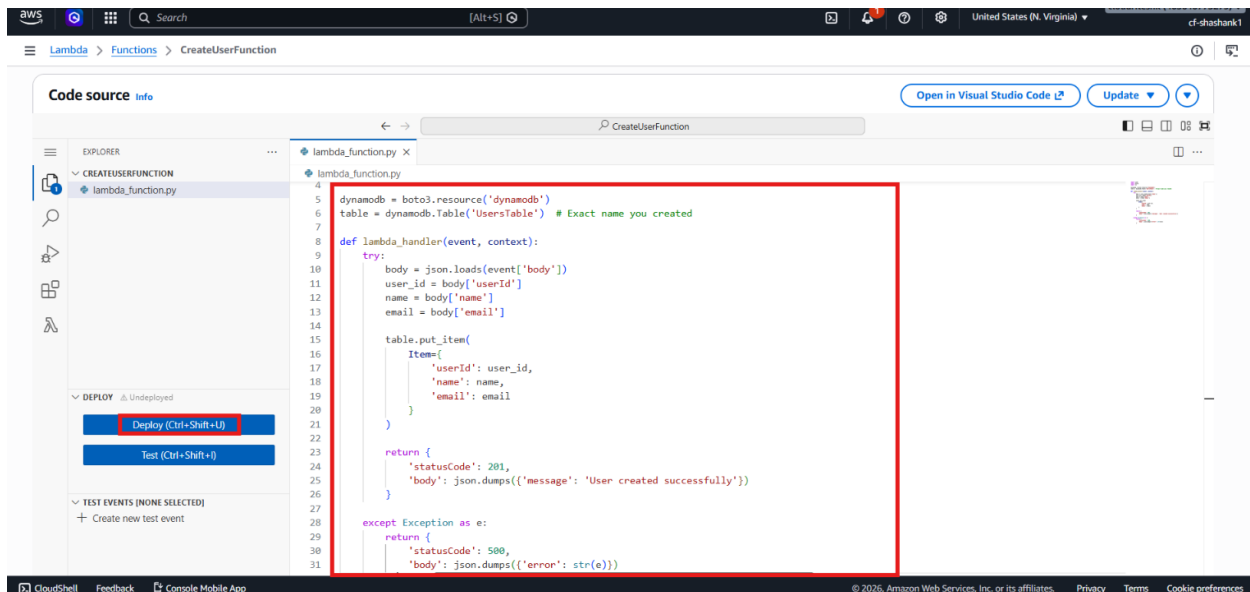
- Here Under **Additional Settings** click on **Custom Execution Role** and Select **LambdaDynamoDBExecutionRole**.



- Click **Create function**.

## Step 7 – Add the Create User Code

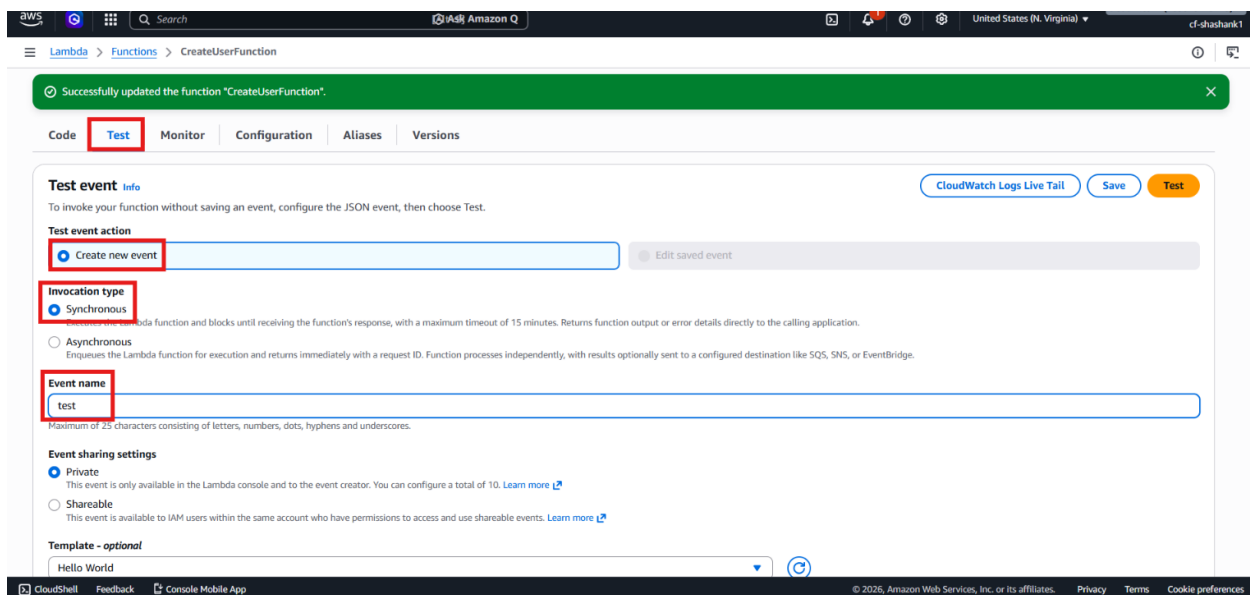
- Open the code editor.
- Replace the default handler code with the provided create-user Lambda code.
- Deploy the function.



- Confirm that the function was updated successfully.

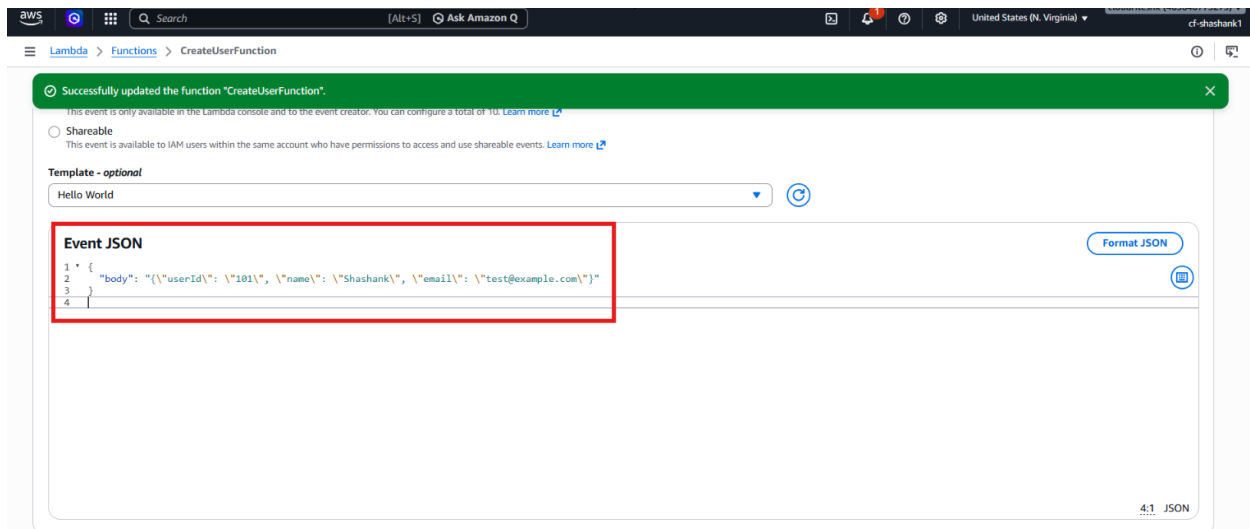
## Step 8 – Test the Create User Function

- Open the **Test** tab.
- Click **Create new test event**.
- Enter a test event name.

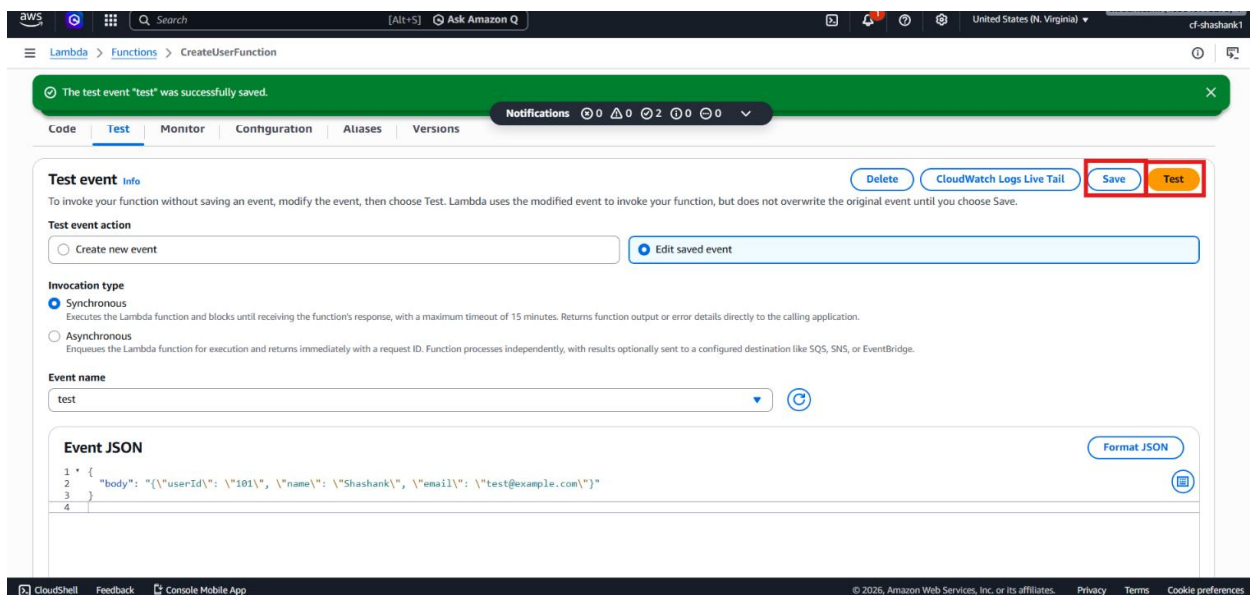


- Use an API Gateway–style payload with the body as a JSON string.

```
{
  "body": "{\"userId\": \"101\", \"name\": \"Shashank\", \"email\": \"test@example.com\"}"
}
```



- Click **Save**.
- Click **Test**.



- Verify that the function returns a success status.
- Open **Amazon DynamoDB**.
- Go to **Tables** → **UsersTable**.
- Click **Explore table items**.



The screenshot shows the AWS DynamoDB console for the 'UsersTable'. The 'Settings' tab is active, showing general information about the table. A red box highlights the 'UsersTable' title in the breadcrumb navigation. Another red box highlights the 'Explore table items' button in the top right corner.

- Confirm that the new record was inserted.

The screenshot shows the AWS DynamoDB console for the 'UsersTable' in the 'Scan or query items' section. The 'Scan' option is selected, and the scan has completed successfully. A green banner indicates 'Completed - Items returned: 1 - Items scanned: 1 - Efficiency: 100% - RCUs consumed: 2'. Below, a table shows the returned item with fields 'userid (String)', 'email', and 'name', and values '101', 'test@exam...', and 'Shashank' respectively. A red box highlights the returned item row.

## Step 9 – Create the Get User Function

- Return to AWS Lambda.
- Click **Create function**.
- Choose **Author from scratch**.
- Enter the function name: GetUserFunction
- Choose **Python 3.12**.

**Create function** [Info](#)

Choose one of the following options to create your function.

☒ **Author from scratch**  
Start with a simple Hello World example.

☐ **Use a blueprint**  
Build a Lambda application from sample code and configuration presets for common use cases.

☐ **Container image**  
Select a container image to deploy for your function.

---

**Basic information**

**Function name**  
Enter a name that describes the purpose of your function.  
  
Function name must be 1 to 64 characters, must be unique to the Region, and can't include spaces. Valid characters are a-z, A-Z, 0-9, hyphens (-), and underscores (\_).

**Runtime** [Info](#)  
Choose the language to use to write your function. Note that the console code editor supports only Node.js, Python, and Ruby.

**Permissions**  
By default, Lambda will create an execution role with permissions to upload logs to Amazon CloudWatch Logs. To use a different role, choose **Custom execution role** under **Additional settings**.

---

**Custom settings** [Info](#)

**Durable execution** - [new](#) [?](#) ☐  
Build resilient, stateful applications with automatic failure recovery.

**EC2 capacity provider** - [new](#) [?](#) ☐  
Run Lambda on your preferred EC2 instance types instead of Lambda's serverless compute (default).

- Under Custom Settings → Additional Settings, enable **ARM64 architecture** to use **x86\_64**.

**Custom settings** [Info](#)

**Durable execution** - [new](#) [?](#) ☐  
Build resilient, stateful applications with automatic failure recovery.

**EC2 capacity provider** - [new](#) [?](#) ☐  
Run Lambda on your preferred EC2 instance types instead of Lambda's serverless compute (default).

▼ **Additional settings**  
Customize your function architecture, execution role, HTTPS endpoints, tenant isolation mode, VPC, code signing, KMS key, and tags.

**General** [Info](#)

**ARM64 architecture** ☒  
Use ARM64 instead of x86\_64 (default).

**Custom execution role** ☐  
Use custom execution role instead of new role with basic Lambda permissions (default).

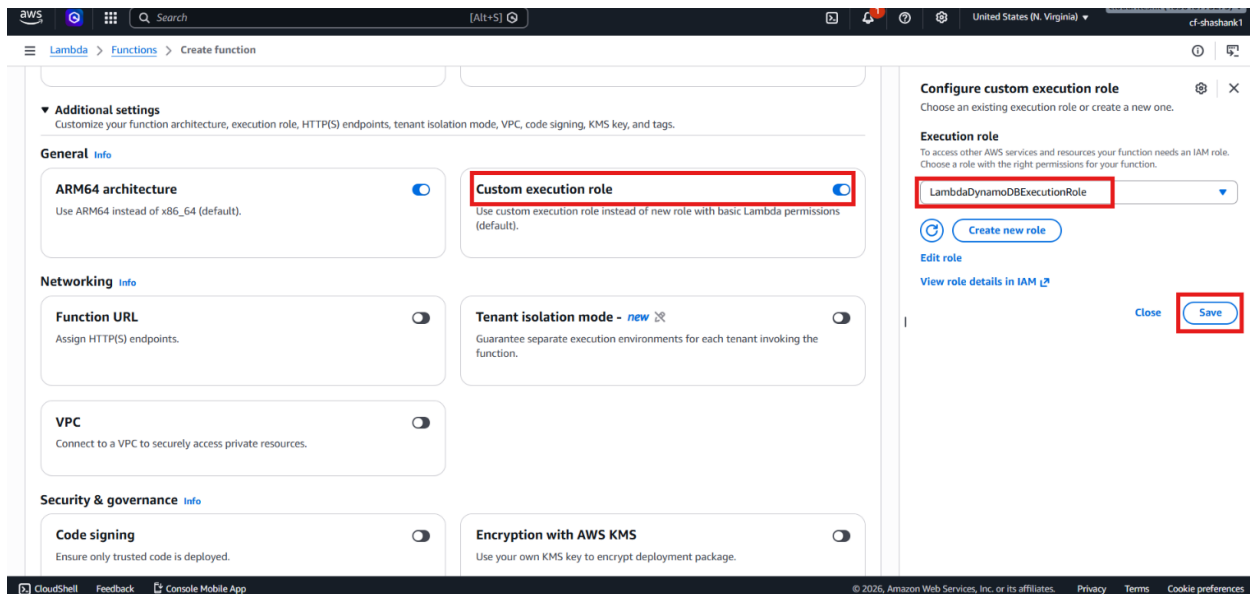
**Networking** [Info](#)

**Function URL** ☐  
Assign HTTPS endpoints.

**Tenant isolation mode** - [new](#) [?](#) ☐  
Guarantee separate execution environments for each tenant invoking the function.

**VPC** ☐  
Connect to a VPC to securely access private resources.

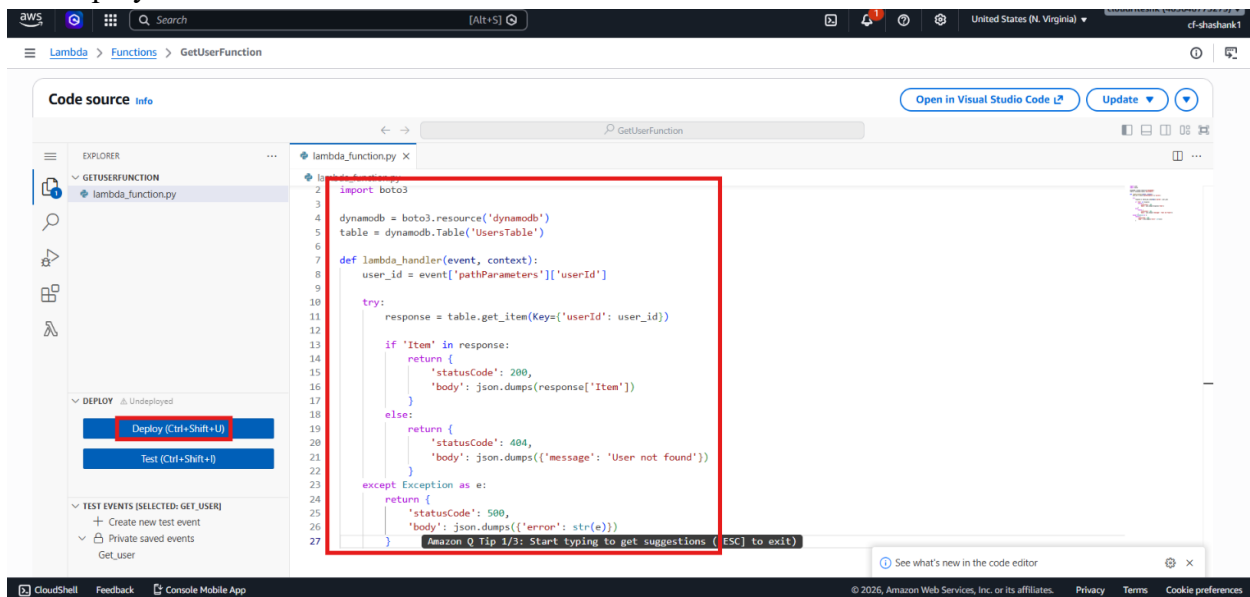
- Here Under **Additional Settings** click on **Custom Execution Role** and use the existing role **LambdaDynamoDBExecutionRole**.



- Click **Create function**.

## Step 10 – Add the Get User Code

- Replace the default code with the provided get\_user Lambda code.
- Deploy the function.



- Open the **Test** tab.

Successfully created the function "GetUserFunction". You can now change its code and configuration. To invoke your function with a test event, choose "Test".

Code **Test** Monitor Configuration Aliases Versions

**Test event** [Info](#) [CloudWatch Logs Live Tail](#) [Save](#) [Test](#)

To invoke your function without saving an event, configure the JSON event, then choose Test.

**Test event action**

☒ Create new event ☐ Edit saved event

**Invocation type**

☒ Synchronous  
Executes the Lambda function and blocks until receiving the function's response, with a maximum timeout of 15 minutes. Returns function output or error details directly to the calling application.

☐ Asynchronous  
Enqueues the Lambda function for execution and returns immediately with a request ID. Function processes independently, with results optionally sent to a configured destination like SQS, SNS, or EventBridge.

**Event name**

Get\_user

Maximum of 25 characters consisting of letters, numbers, dots, hyphens and underscores.

**Event sharing settings**

☒ Private  
This event is only available in the Lambda console and to the event creator. You can configure a total of 10. [Learn more](#)

☐ Shareable  
This event is available to IAM users within the same account who have permissions to access and use shareable events. [Learn more](#)

**Template - optional**

Hello World

- Create a new test event using the path parameter structure.

```
{
  "pathParameters": {
    "userId": "101"
  }
}
```

Successfully created the function "GetUserFunction". You can now change its code and configuration. To invoke your function with a test event, choose "Test".

This event is only available in the Lambda console and to the event creator. You can configure a total of 10. [Learn more](#)

☐ Shareable  
This event is available to IAM users within the same account who have permissions to access and use shareable events. [Learn more](#)

**Template - optional**

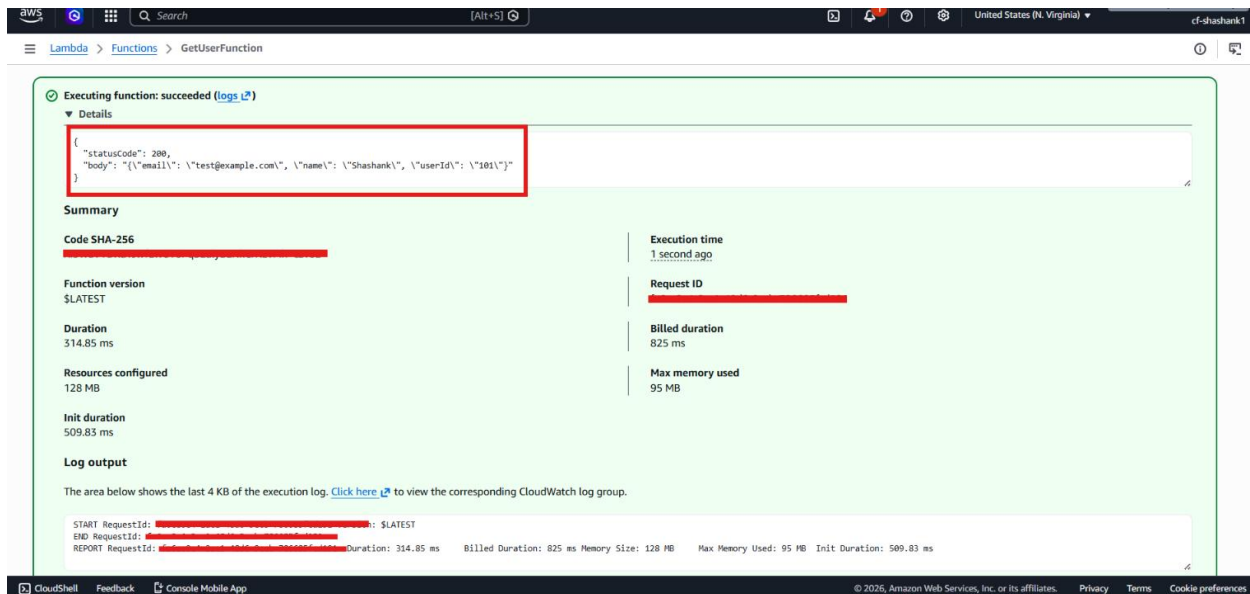
Hello World

**Event JSON** [Format JSON](#)

```
1 {
2   "pathParameters": {
3     "userId": "101"
4   }
5 }
```

6:1 JSON

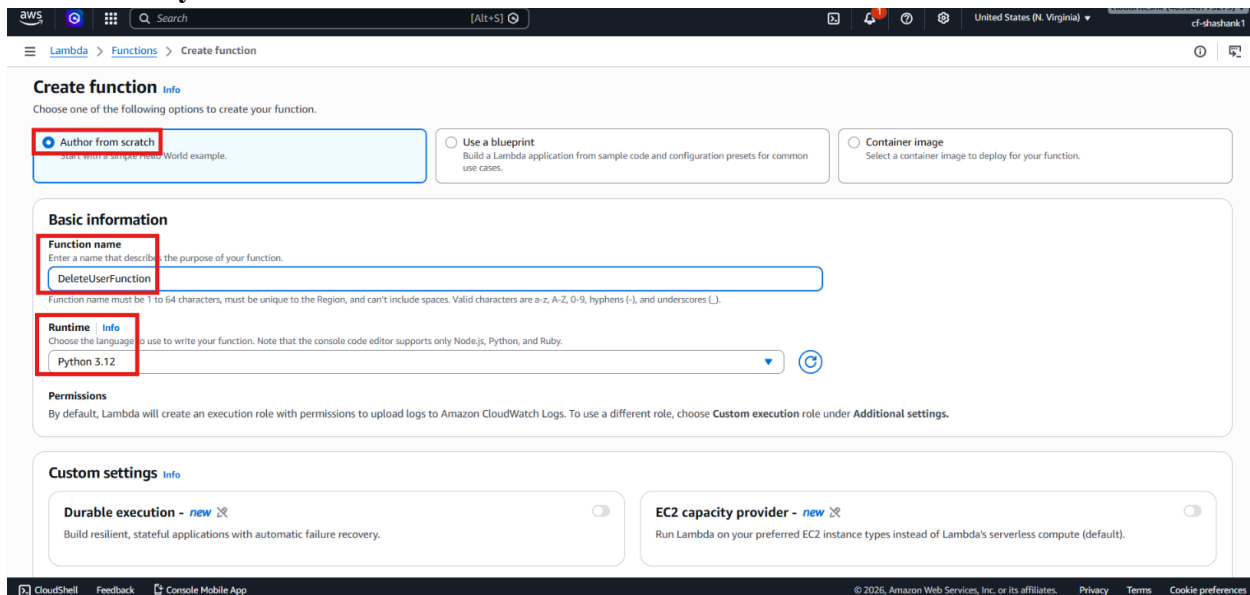
- Click **Save**.
- Click **Test**.



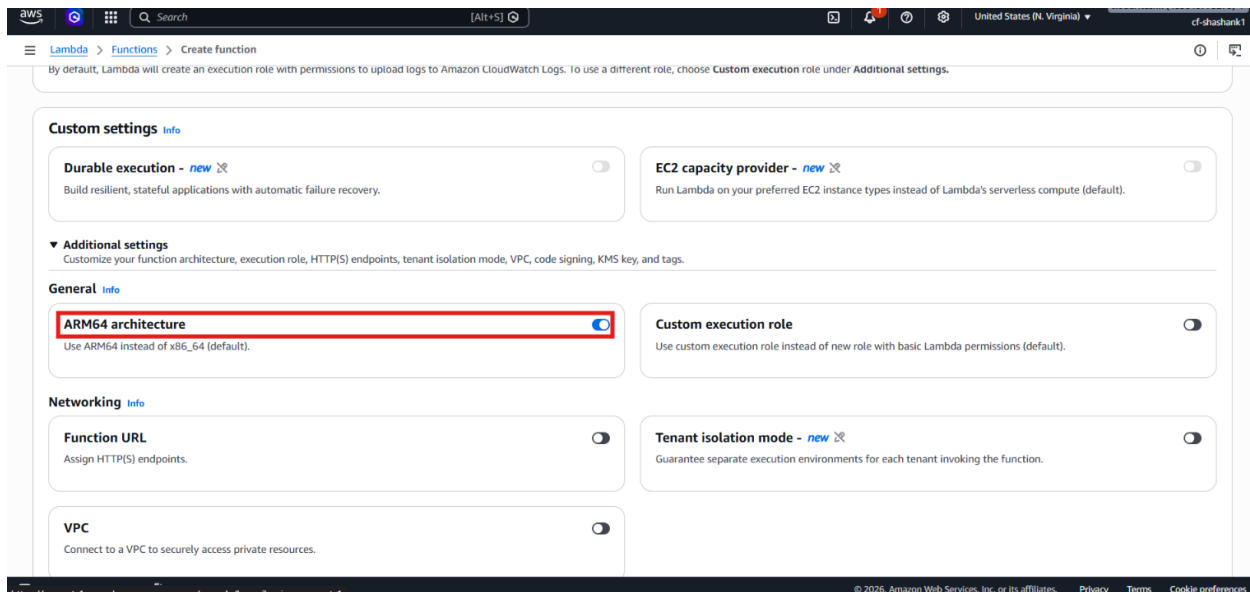
- Verify that the function returns the user details if the record exists.
- Change the userId to a missing value such as 102.
- Test again.
- Confirm that the function returns **User not found** for a missing record.

## Step 11 – Create the Delete User Function

- Return to AWS Lambda.
- Click **Create function**.
- Choose **Author from scratch**.
- Enter the function name: DeleteUserFunction
- Choose **Python 3.12**.



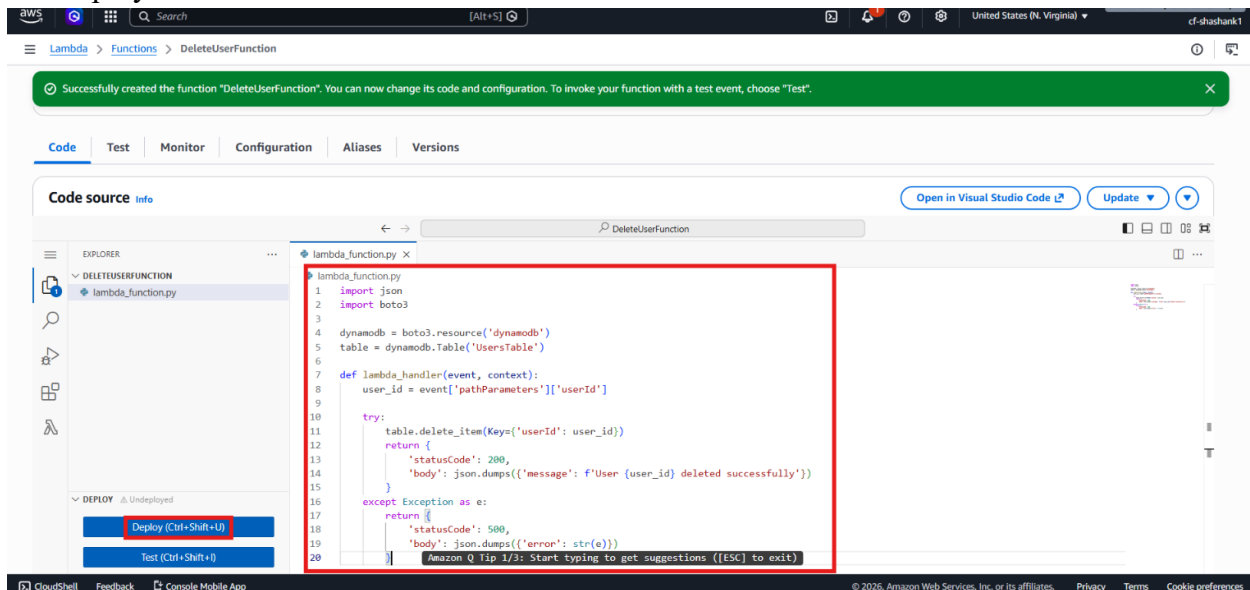
- Under Custom Settings → Additional Settings, enable **ARM64 architecture** to use **x86\_64**.



- Here Under **Additional Settings** click on **Custom Execution Role** and use the existing role **LambdaDynamoDBExecutionRole**.
- Click **Create function**.

## Step 12 – Add the Delete User Code

- Replace the default code with the provided delete-user Lambda code.
- Deploy the function.



- Open the **Test** tab.

Successfully updated the function "DeleteUserFunction".

**Test event** [Info](#)

To invoke your function without saving an event, configure the JSON event, then choose Test.

**Test event action**

☒ Create new event ☐ Edit saved event

**Invocation type**

☒ Synchronous  
Executes the Lambda function and blocks until receiving the function's response, with a maximum timeout of 15 minutes. Returns function output or error details directly to the calling application.

☐ Asynchronous  
Enqueues the Lambda function for execution and returns immediately with a request ID. Function processes independently, with results optionally sent to a configured destination like SQS, SNS, or EventBridge.

**Event name**

delete\_user

Maximum of 25 characters consisting of letters, numbers, dots, hyphens and underscores.

**Event sharing settings**

☒ Private  
This event is only available in the Lambda console and to the event creator. You can configure a total of 10. [Learn more](#)

☐ Shareable  
This event is available to IAM users within the same account who have permissions to access and use shareable events. [Learn more](#)

**Template - optional**

Hello World

**Event JSON** [Format JSON](#)

- Create a new test event using the same path parameter structure.

```
{
  "pathParameters": {
    "userId": "101"
  }
}
```

Successfully updated the function "DeleteUserFunction".

Executes the Lambda function and blocks until receiving the function's response, with a maximum timeout of 15 minutes. Returns function output or error details directly to the calling application.

☐ Asynchronous  
Enqueues the Lambda function for execution and returns immediately with a request ID. Function processes independently, with results optionally sent to a configured destination like SQS, SNS, or EventBridge.

**Event name**

delete\_user

Maximum of 25 characters consisting of letters, numbers, dots, hyphens and underscores.

**Event sharing settings**

☒ Private  
This event is only available in the Lambda console and to the event creator. You can configure a total of 10. [Learn more](#)

☐ Shareable  
This event is available to IAM users within the same account who have permissions to access and use shareable events. [Learn more](#)

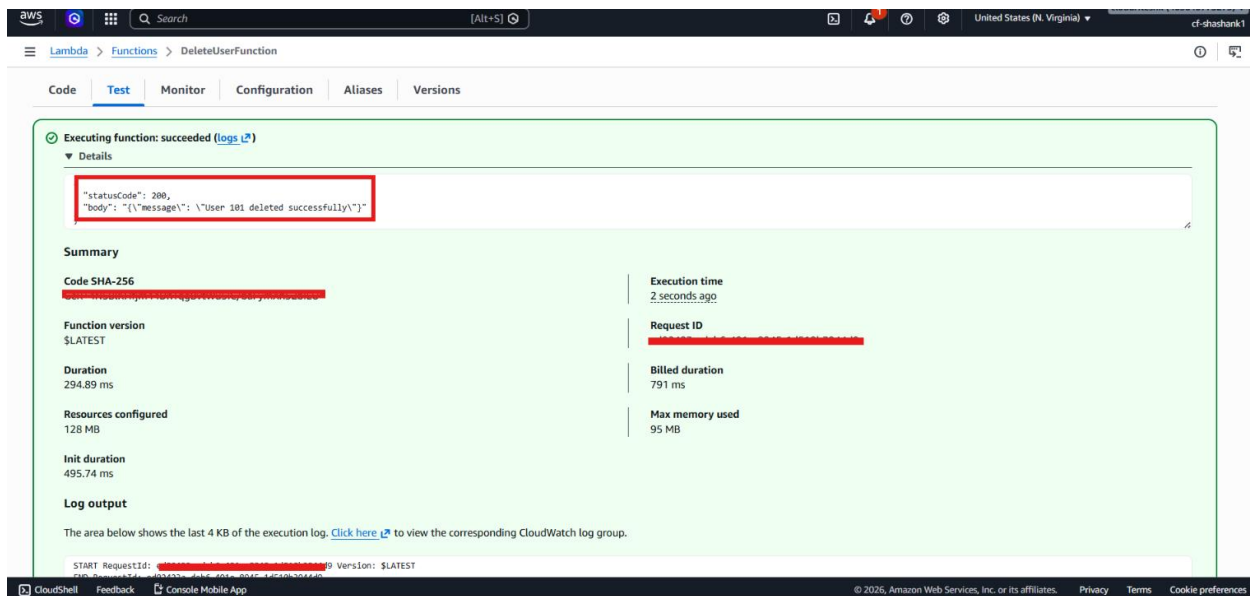
**Template - optional**

Hello World

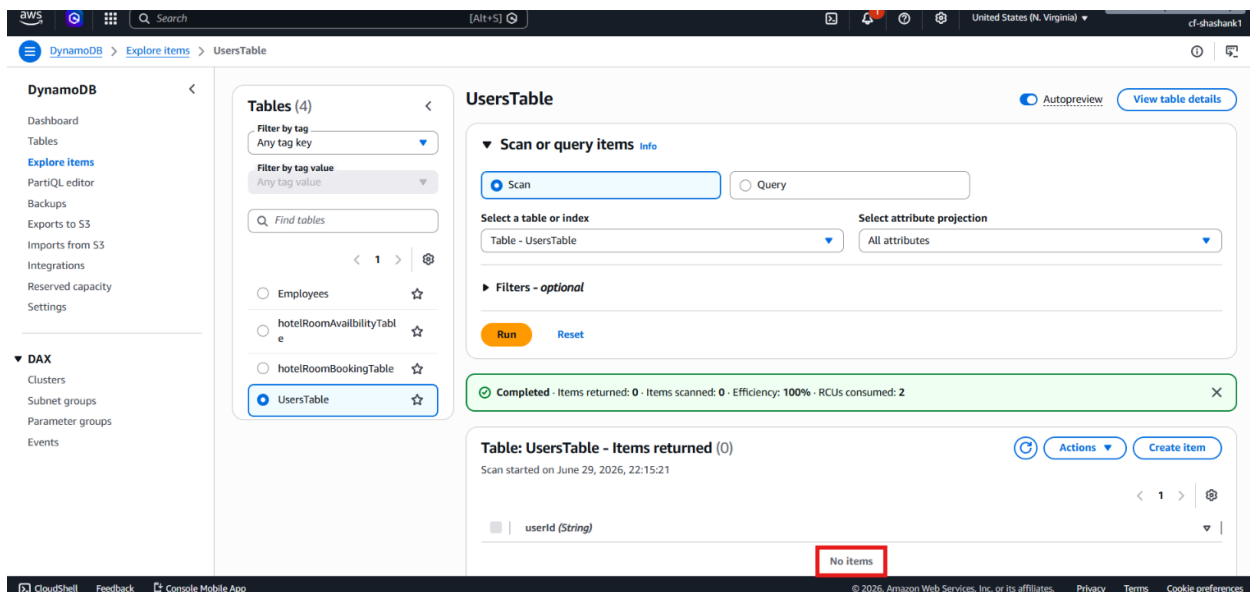
**Event JSON** [Format JSON](#)

```
1 {
2   "pathParameters": {
3     "userId": "101"
4   }
5 }
6
```

- Click **Save**.
- Click **Test**.



- Verify that the function returns a deletion success message.
- Open **Amazon DynamoDB**.
- Refresh **UsersTable**.



- Confirm that the record was removed.

## Step 13 – Confirm the CRUD Backend Is Ready

- Verify that all three Lambda functions exist:
  - createUserFunction
  - getUserFunction
  - deleteUserFunction
- Confirm that all three functions use the same **IAM role**.



- Confirm that the DynamoDB table has been tested successfully with create, read, and delete operations.
- Prepare for **API Gateway** integration in the next part of the lab.

## Verification

- Verified that the **UsersTable** was created successfully in **Amazon DynamoDB** with `userId` as the partition key.
- Verified that the **LambdaDynamoDBExecutionRole** was created and attached to all three **AWS Lambda** functions.
- Verified that **CreateUserFunction** successfully inserted a new user record into the DynamoDB table.
- Verified that **GetUserFunction** returned the correct user details for an existing `userId`.
- Verified that **GetUserFunction** returned **"User not found"** for a non-existent `userId`.
- Verified that **DeleteUserFunction** successfully removed the user record from the DynamoDB table.
- Verified that the DynamoDB table reflected the create and delete operations correctly.
- Verified that the CRUD backend was fully functional and ready for **Amazon API Gateway** integration.